REMOVING TOKENIZATION BIAS AT INFERENCE TIME WITH COVER ENCODINGS

Anonymous authors
Paper under double-blind review

ABSTRACT

Tokenized language models (LMs) predict the next token given one deterministic encoding of the prompt. We show that this induces *tokenization bias*: a discrepancy between the model's next-token predictions and the byte-level distribution the same model implicitly defines, which can assign zero probability to the correct continuation when a prompt ends in the middle of a token, as routinely happens in fill-in-the-middle (FIM) code completion. We prove that the probability of any byte prefix under a tokenized LM equals a sum over its *cover encodings*, the valid token sequences whose decoding starts with that prefix, and derive an inference-time algorithm that finds them by a backward search and samples the next byte with one model call per byte. This turns any tokenized LM into a statistically equivalent token-free model without retraining. On Markov chains the method recovers the true transition probabilities that the naive tokenized estimate distorts. On HumanEval random-span infilling with CodeLlama-7b, it raises the greedy pass rate in the suffix-prefix-middle format from 45.0% to 63.9%, ahead of token healing (62.7%) and token alignment (63.0%). Because all models share the byte-level output space, models with different tokenizers can be ensembled by averaging: the top-2 byte-level ensemble outperforms its best member on MMLU, PIQA, NQ, TriviaQA and HumanEval, but not when members differ widely in ability, as on GSM8K.

1 INTRODUCTION

Modern large language models (LLMs) do not operate on raw text. A tokenizer, typically byte-pair encoding (BPE) (Sennrich et al., 2016; Wang et al., 2020) or a related subword scheme (Kudo & Richardson, 2018), first maps the byte string to a sequence of tokens from a fixed vocabulary, and the Transformer (Vaswani et al., 2017) models the distribution of the next token. Tokenization shortens sequences and has been central to the success of open models such as Llama 2 (Touvron et al., 2023), Mistral (Jiang et al., 2023a), Yi (Young et al., 2024) and Code Llama (Rozière et al., 2023). It also makes the model's predictions depend on an artifact of preprocessing: the particular segmentation of the context.

This dependence becomes visible whenever a prompt ends at a position that does not coincide with a token boundary of the text the model would have seen during training. Figure 1 (left) shows a code completion prompt ending in `return 5 + a`, whose correct continuation is the character `a`, completing the argument name `aa`. Because the tokenizer would have encoded `aa` as a single token, the model has learned that the token `a` is never followed by another `a`, and assigns probability 0.0 to the correct next character. We call this discrepancy between the conditional distribution of a tokenized model given one deterministic encoding of the prompt and the underlying byte-level distribution *tokenization bias*. It is not a rare corner case: in fill-in-the-middle (FIM) code completion (Bavarian et al., 2022; Fried et al., 2023), the boundary between the given prefix and the missing middle may fall anywhere, including inside a word. Existing remedies such as token healing (Dagan et al., 2024) and token alignment (Athiwaratkun et al., 2024) back up over the last token(s) of the prompt and constrain generation to match the removed characters; they are effective heuristics but do not compute the probabilities the model actually assigns to bytes.

A second consequence of tokenization is that models with different vocabularies produce distributions over different, incompatible output spaces. Averaging next-token probabilities, the simplest en-

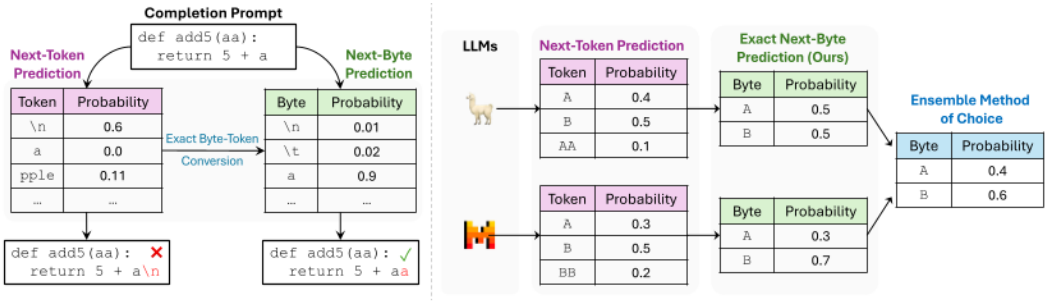


Figure 1: Left: when a prompt ends mid-token, as in this schematic code completion example, a tokenized LM suffers from tokenization bias: the correct continuation `a` (completing the argument name `aa`) receives zero next-token probability. Converting the same model's next-token predictions into exact next-byte predictions assigns it high probability. Right: mapping the next-token predictions of LMs with different vocabularies to statistically equivalent next-byte predictions (Section 3) puts them in a common space, where any ensemble method, such as averaging or a mixture of experts, can be applied.

semble, is therefore not possible across the many open-weight models that use different tokenizers, and existing heterogeneous ensembles resort to output reranking or learned vocabulary alignments (Jiang et al., 2023b; Huang et al., 2024; Yu et al., 2024).

Our central observation is that tokenization bias is an artifact of conditioning on a single tokenization of the past. Any tokenized LM defines a distribution over byte strings, and the probability of a byte prefix is obtained by marginalizing over all token sequences that could have produced it. We show that this marginal is a finite sum over a small set of *cover encodings*, and that the resulting byte-level next-symbol distribution can be computed exactly and efficiently at inference time (Figure 1, right). Our contributions are:

- **A formal account of tokenization bias.** We define valid and invalid encodings for deterministic tokenizers, show on Markov chains how conditioning on one encoding yields zero or distorted transition probabilities, and state a byte-token representation lemma that expresses the probability of any byte prefix through token-level probabilities (Section 3).
- **An exact, training-free byte-level sampler.** We give a backward search for cover encodings and a recursive next-byte sampling procedure that requires one model evaluation per generated byte, converting any tokenized LM into a statistically equivalent token-free model (Sections 3.4–3.5).
- **Robust fill-in-the-middle.** On the HumanEval random-span benchmark with CodeLlama-7b, byte-level prediction removes most of the drop caused by the suffix-prefix-middle prompt format (greedy pass rate 45.0% $\rightarrow$ 63.9%) and scores slightly above token healing and token alignment (Section 4.3).
- **Ensembling across tokenizers.** By mapping models into a shared byte space we average the predictions of models with different vocabularies; the top-2 ensemble exceeds the best member on most knowledge, reasoning and coding benchmarks, and we characterize when it does not (Section 4.4).

2 Related Work

Tokenization and its side effects. Subword tokenizers such as BPE (Sennrich et al., 2016), its byte-level variant (Wang et al., 2020; Radford et al., 2019) and the unigram model of SentencePiece (Kudo & Richardson, 2018; Kudo, 2018) are the default front end of LLMs. A growing body of work documents that the choice of tokenizer affects downstream quality (Bostrom & Durrett, 2020; Rust et al., 2021; Ali et al., 2023; Schmidt et al., 2024), arithmetic (Singh & Strouse, 2024) and the statistical properties of the learned model (Zouhar et al., 2023; Gastaldi et al., 2024). Stochastic

tokenization during training (Kudo, 2018; Provilkov et al., 2020) exposes models to multiple segmentations, but standard LLMs are trained with a single deterministic tokenizer, which is the setting we study. Cao & Rimell (2021) and Chirkova et al. (2023) argue for evaluating LMs by the marginal likelihood over tokenizations of a full string and approximate it by sampling; we instead compute exact marginal probabilities of byte *prefixes*, which is what next-symbol prediction from an arbitrary prompt requires, and exploit the determinism of the tokenizer to restrict the sum to a small set of valid encodings.

Prompt boundaries in code completion. FIM training (Bavarian et al., 2022; Fried et al., 2023) lets code models such as Code Llama (Rozière et al., 2023) and StarCoder (Li et al., 2023) complete a missing middle given a prefix and a suffix, in either the prefix-suffix-middle (PSM) or the suffix-prefix-middle (SPM) format. When the prefix ends inside a token, generation starts from a segmentation the model never saw in training. Token healing (Dagan et al., 2024) removes the last prompt token and constrains the first generated token to start with the removed characters; token alignment (Athiwaratkun et al., 2024) backtracks over several tokens and constrains generation by character matching. Both are heuristics that change the conditioning context; our method instead computes the exact byte-level distribution, summing over all segmentations of the prompt end that the model could have produced, including the backed-up segmentations these heuristics consider.

Token-free models. Byte- and character-level models avoid tokenization altogether (Xue et al., 2022; Clark et al., 2022; Tay et al., 2022; Yu et al., 2023), at the cost of longer sequences and of training new models from scratch. Our approach is complementary: it obtains byte-level predictions from existing tokenized models without any training. Zero-shot tokenizer transfer (Minixhofer et al., 2024) replaces a model's tokenizer by predicting new embeddings, whereas we keep the model and tokenizer unchanged.

Ensembling LLMs. Ensembles and mixtures of experts (Shazeer et al., 2017; Jiang et al., 2024) and weight averaging (Wortsman et al., 2022) combine models that share an architecture or a vocabulary. For heterogeneous LLMs, LLM-Blender (Jiang et al., 2023b) ranks and fuses complete outputs, while DeePEn (Huang et al., 2024) and GaC (Yu et al., 2024) align the next-token distributions of different vocabularies through relative representations or vocabulary mappings. Mapping each model to exact next-byte probabilities gives a common output space without learned alignment, so ensemble members can be combined at every generation step by simple averaging.

3 METHODOLOGY

3.1 SETUP AND NOTATION

Let $\mathcal{A}$ be the alphabet of bytes and $x_1^n = x_1 x_2 \dots x_n \in \mathcal{A}^n$ a byte string, with x_i^j denoting the substring from position i to j. A tokenizer has a vocabulary $\mathcal{V} \subset \mathcal{A}^+$ of byte strings that contains every single byte, a deterministic encoding function $\mathrm{encode} : \mathcal{A}^* \to \mathcal{V}^*$ (e.g., BPE, or maximal prefix encoding (MPE), which greedily takes the longest matching token), and a decoding function $\mathrm{decode} : \mathcal{V}^* \to \mathcal{A}^*$ that concatenates tokens, with $\mathrm{decode}(\mathrm{encode}(x_1^n)) = x_1^n$. We write $\vec{t} = t_1^k = (t_1, \dots, t_k)$ for token sequences, $|t|$ for the number of bytes of a token and $L = \max_{t \in \mathcal{V}} |t|$ for the maximal token length.

A tokenized LM specifies next-token probabilities $P(t_{k+1} \mid t_1^k)$ and generates an unbounded token stream $t_1, t_2, \dots$, which in turn defines a byte stream $\mathrm{decode}(t_1)\,\mathrm{decode}(t_2) \cdots$. For a token prefix we write $P(\vec{t}) = \prod_{i=1}^{k} P(t_i \mid t_1^{i-1})$ for the probability that the generated token stream begins with $\vec{t}$, and for a byte string we write $P(x_1^n)$ for the probability that the generated byte stream begins with x_1^n. Our goal is the byte-level next-symbol distribution

$$P(x_{n+1} \mid x_1^n) = \frac{P(x_1^{n+1})}{P(x_1^n)}. \tag{1}$$

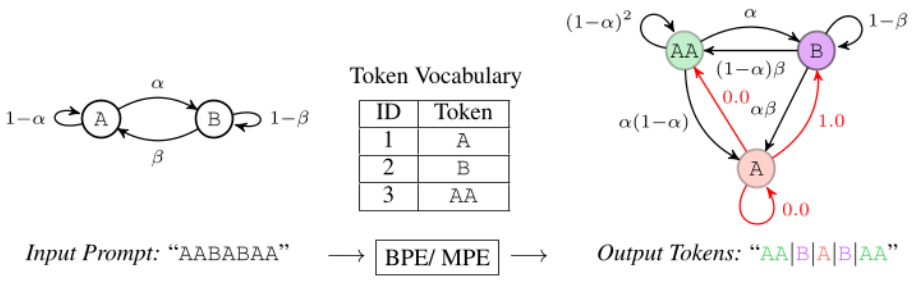


Figure 2: Tokenization bias on a first-order Markov chain over the bytes A and B (left), tokenized with the vocabulary {A, B, AA} by BPE or MPE (middle). Because two consecutive A bytes are always merged into the token AA, a standalone token A is always followed by B in encoded text. Given the context token A, the tokenized model therefore assigns probability 0 to the next token being A or AA and probability 1 to B (red edges, right), whereas in the original chain the byte after A is A with probability $1 - \alpha$. This occurs whenever a prompt ends with a token that is a prefix of another token, as is common in fill-in-the-middle (FIM) tasks.

3.2 Valid encodings and tokenization bias

During training, a tokenized LM only ever sees token sequences produced by encode. A deterministic tokenizer never produces certain token sequences, because its merge (or longest-match) rules would have segmented the same bytes differently.

Definition 1 (Valid and invalid encodings)**.** *A token sequence $\vec{t}$ is a* valid encoding *if* $\mathrm{encode}(\mathrm{decode}(\vec{t})) = \vec{t}$*, and an* invalid encoding *otherwise.*

For example, with vocabulary $\{\mathtt{A}, \mathtt{B}, \mathtt{AA}\}$, the sequence $(\mathtt{A}, \mathtt{A})$ is invalid since $\mathrm{encode}(\mathtt{AA}) = (\mathtt{AA})$. A model that fits its training data well therefore learns to assign (near) zero probability to invalid encodings; we make this explicit.

Assumption 1. *The tokenized LM assigns zero probability to invalid encodings: $P(\vec{t}) > 0$ only if $\vec{t}$ is valid.*

The standard way to continue a prompt x_1^n is to condition on its single encoding, i.e., to use $P(t_{k+1} \mid \mathrm{encode}(x_1^n))$. We call the discrepancy between this conditional and the byte-level distribution of Equation 1 *tokenization bias*. Figure 2 illustrates it on a first-order Markov chain over $\{\mathtt{A}, \mathtt{B}\}$ with $P(\mathtt{A} \mid \mathtt{A}) = 1 - \alpha$, tokenized with the vocabulary above. Whenever the encoded context ends with the single-byte token A, the next byte cannot be A, because two consecutive As would have been merged into AA; the tokenized model hence predicts $P(t_{k+1} = \mathtt{A} \mid \ldots, \mathtt{A}) = 0$ although the true probability of the next byte being A is $1 - \alpha$. The token-level model is not wrong (it correctly models the token process), but conditioning on $\mathrm{encode}(x_1^n)$ implicitly conditions on the event that the *next* token starts exactly at position $n + 1$, which is information the byte string does not contain. The same effect occurs whenever a prompt ends with a token that is a prefix of another token, as in Figure 1 and in FIM prompts.

3.3 The byte-token representation lemma

To remove the bias we marginalize over every token sequence that could have produced the observed bytes. Since the last token of such a sequence may extend beyond position n, we define the following set.

Definition 2 (Cover encodings)**.** *The* cover encodings *of x_1^n are the valid token sequences $\vec{t} = t_1^k$ such that $\mathrm{decode}(\vec{t})$ begins with x_1^n and $\mathrm{decode}(t_1^{k-1})$ is a strict prefix of x_1^n, i.e., the last token t_k starts at a position $\leq n$ and ends at a position $\geq n$. We denote this set by $\mathrm{cover}(x_1^n)$.*

Lemma 1 (Byte-token representation)**.** *Under Assumption 1, for every byte string x_1^n,*

$$P(x_1^n) = \sum_{\vec{t} \in \mathrm{cover}(x_1^n)} P(\vec{t}). \tag{2}$$

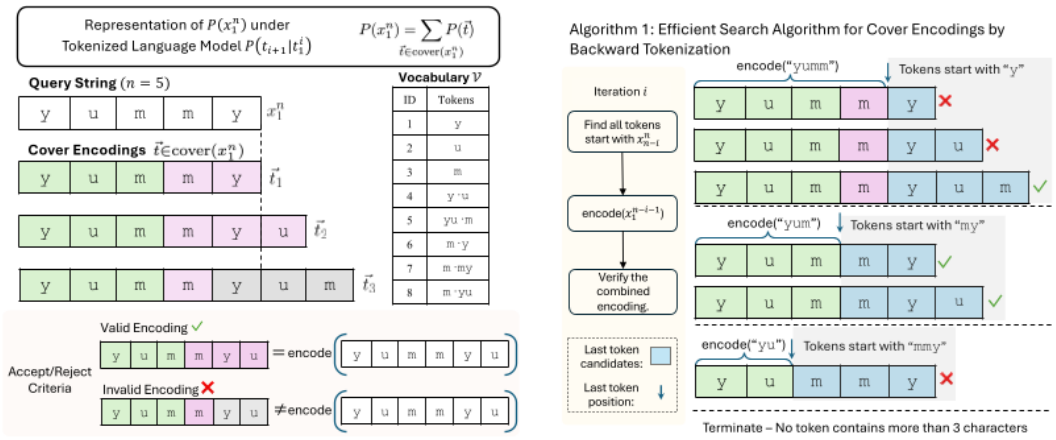


Figure 3: Left: the probability $P(x_1^n)$ of a byte string under a tokenized LM is the sum of the probabilities $P(\vec{t})$ of its cover encodings $\vec{t} \in \text{cover}(x_1^n)$, shown for the query string $\texttt{yummy}$ ($n = 5$) and an eight-token vocabulary. An encoding is valid if re-encoding the bytes it decodes to returns the same tokens, and invalid otherwise (Definition 1). Right: Algorithm 1 finds them by backward tokenization: at iteration i it lists the tokens that start with the last i bytes, encodes the remaining prefix, and keeps the combined encoding only if it is valid (green tick; red cross: invalid). The figure counts iterations from zero. The search stops once the suffix is longer than every token (here, three characters). The example uses BPE.

Proof. Consider any infinite token stream whose byte stream begins with x_1^n. The lengths $|\,\text{decode}(t_1^j)|$ increase strictly with j, so there is a unique k with $|\,\text{decode}(t_1^{k-1})| < n \leq |\,\text{decode}(t_1^k)|$; the prefix t_1^k then decodes to a string beginning with x_1^n and $\text{decode}(t_1^{k-1})$ is a strict prefix of x_1^n. Conversely, every stream that begins with such a prefix has a byte stream beginning with x_1^n. The event "the byte stream begins with x_1^n" is therefore the disjoint union, over all token sequences satisfying the prefix conditions of Definition 2, of the events "the token stream begins with $\vec{t}$". Summing their probabilities and dropping the invalid sequences, which have probability zero by Assumption 1, gives Equation 2. □

Figure 3 (left) shows the cover encodings of the string $\texttt{yummy}$ under a small BPE vocabulary. Combining Lemma 1 with Equation 1 gives the exact next-byte distribution of the tokenized LM. Its cost depends only on the number of cover encodings, which is small: a cover encoding t_1^k of positive probability satisfies $P(t_1^{k-1}) \geq P(t_1^k) > 0$, so t_1^{k-1} is valid and therefore equals $\text{encode}(x_1^j)$ for the position $j = |\,\text{decode}(t_1^{k-1})| \in [n - L, n - 1]$ at which the last token starts. Cover encodings of probability zero do not contribute to Equation 2 and can be ignored; a cover encoding of positive probability is thus fully determined by the start position j of its last token and by that token, which must begin with x_{j+1}^n.

3.4 Finding cover encodings by backward search

Enumerating all token sequences that decode to strings starting with x_1^n is intractable, but the observation above suggests searching backwards from the end of the string. For $i = 1, 2, \ldots$, we let the last token start i bytes before the end, at position $n-i+1$: the history is then fixed to $\text{encode}(x_1^{n-i})$, and the candidate last tokens are the tokens whose first i bytes equal x_{n-i+1}^n. Each candidate sequence is kept if it is a valid encoding. One forward pass of the LM on $\text{encode}(x_1^{n-i})$ yields both $P(\text{encode}(x_1^{n-i}))$ (accumulated along the pass) and the probabilities $P(t \mid \text{encode}(x_1^{n-i}))$ of all candidate last tokens. The search terminates once i exceeds L, as no token is longer than L bytes (Figure 3, right). Algorithm 1 summarizes the procedure, which needs at most L forward passes, one per distinct history; since $\text{encode}(x_1^{n-i})$ is in general not a prefix of $\text{encode}(x_1^n)$, the histories are evaluated separately, but they can be batched and share the key-value cache of their common leading tokens (Appendix A).

Algorithm 1 Search for cover encodings by backward tokenization

Require: byte string x_1^n, tokenizer $(\mathcal{V}, \mathrm{encode}, \mathrm{decode})$, tokenized LM P, maximal token length L

$\mathcal{C} \leftarrow \emptyset$
for $i = 1, \ldots, \min(L, n)$ **do**
  $\boldsymbol{t}_{\mathrm{hist}} \leftarrow \mathrm{encode}(x_1^{n-i})$ ▷ fixed history; empty if $i = n$
  Compute $P(\boldsymbol{t}_{\mathrm{hist}})$ and $P(\cdot \mid \boldsymbol{t}_{\mathrm{hist}})$ with one forward pass
  for each $t \in \mathcal{V}$ whose first i bytes equal x_{n-i+1}^n **do**
    if $(\boldsymbol{t}_{\mathrm{hist}}, t)$ is a valid encoding **then** ▷ $\mathrm{encode}(\mathrm{decode}(\boldsymbol{t}_{\mathrm{hist}}, t)) = (\boldsymbol{t}_{\mathrm{hist}}, t)$
      $\mathcal{C} \leftarrow \mathcal{C} \cup \{(\boldsymbol{t}_{\mathrm{hist}}, t)\}$ with $P(\boldsymbol{t}_{\mathrm{hist}}, t) = P(\boldsymbol{t}_{\mathrm{hist}})\, P(t \mid \boldsymbol{t}_{\mathrm{hist}})$
    end if
  end for
end for
return $\mathcal{C}$, all cover encodings of x_1^n with positive probability, with their probabilities $P(\vec{t})$

3.5 Sampling the next byte

Algorithm 1 is only needed once, for the prompt. During generation, the cover encodings of x_1^{n+1} are obtained recursively from those of x_1^n, at the cost of a single model call per byte (Figure 4).

Proposition 1. *Under Assumption 1, discarding encodings of probability zero,* $\mathrm{cover}(x_1^n b)$ *for a byte* b *is the disjoint union of*

$$\mathcal{C}_{n+1}(b) = \{\vec{t} \in \mathrm{cover}(x_1^n) : |\,\mathrm{decode}(\vec{t})| > n \text{ and the } (n{+}1)\text{-th byte of } \mathrm{decode}(\vec{t}) \text{ is } b\}, \tag{3}$$

$$\bar{\mathcal{C}}_{n+1}(b) = \{(\mathrm{encode}(x_1^n), t) : t \in \mathcal{V} \text{ begins with } b \text{ and } (\mathrm{encode}(x_1^n), t) \text{ is valid}\}. \tag{4}$$

Proof. Let $t_1^k \in \mathrm{cover}(x_1^n b)$. If its last token starts at or before position n, then $\mathrm{decode}(t_1^{k-1})$ is a strict prefix of x_1^n and $t_1^k \in \mathrm{cover}(x_1^n)$ extends beyond position n with b at position $n+1$, i.e., $t_1^k \in \mathcal{C}_{n+1}(b)$. Otherwise the last token starts at position $n+1$, so $\mathrm{decode}(t_1^{k-1}) = x_1^n$; since $P(t_1^{k-1}) \geq P(t_1^k) > 0$, Assumption 1 implies that t_1^{k-1} is valid, hence $t_1^{k-1} = \mathrm{encode}(x_1^n)$ and $t_1^k \in \bar{\mathcal{C}}_{n+1}(b)$. The converse inclusions follow directly from Definition 2. □

By Lemma 1 and Proposition 1, the next-byte distribution is

$$P(x_{n+1} = b \mid x_1^n) = \frac{1}{P(x_1^n)} \Big[\sum_{\vec{t} \in \mathcal{C}_{n+1}(b)} P(\vec{t}) + P(\mathrm{encode}(x_1^n)) \sum_{t:\, (\mathrm{encode}(x_1^n), t) \in \bar{\mathcal{C}}_{n+1}(b)} P(t \mid \mathrm{encode}(x_1^n)) \Big]. \tag{5}$$

The first sum regroups previously computed cover encodings by the byte they predict at position $n+1$ and requires no model call. The second sum requires one forward pass on $\mathrm{encode}(x_1^n)$, the canonical encoding of the current string, whose probability is available because $\mathrm{encode}(x_1^n)$ is the unique cover encoding of x_1^n that decodes to exactly n bytes (when it has positive probability). After sampling x_{n+1}, we keep $\mathrm{cover}(x_1^{n+1}) = \mathcal{C}_{n+1}(x_{n+1}) \cup \bar{\mathcal{C}}_{n+1}(x_{n+1})$ and repeat. In practice we store probabilities relative to $P(x_1^n)$ in log space to avoid underflow. The procedure is a decoding wrapper: it changes neither the weights nor the tokenizer, and sampling from Equation 5 byte by byte produces byte strings with exactly the distribution of the tokenized LM's output, while allowing the prompt to end at an arbitrary byte. Greedy decoding takes the most probable byte at each step. The procedure extends naturally to beam search over bytes, keeping the B byte strings with the highest cumulative byte-level log-probability, each with its own set of cover encodings (Appendix A; our logs record only the beam widths used).

3.6 Ensembling models with different tokenizers

Equation 5 maps every tokenized LM, whatever its vocabulary, to a distribution over the same next-byte space $\mathcal{A}$. Given M models $P_1, \ldots, P_M$, each maintaining its own cover encodings for the

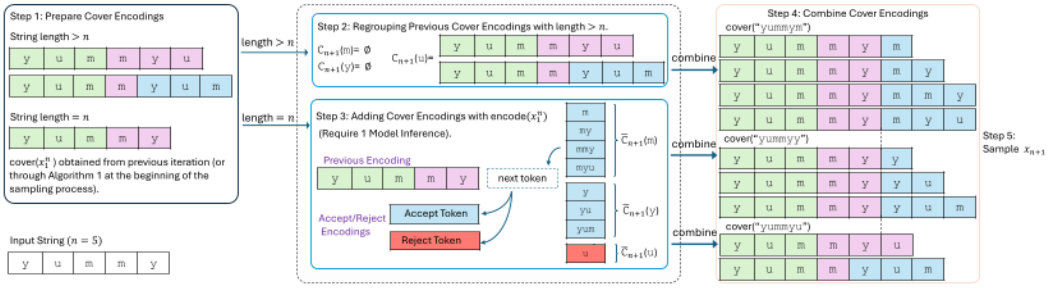


Figure 4: Sampling the next byte x_{n+1} from a tokenized LM $P(t_{k+1} \mid t_1^k)$, continuing the example of Figure 3. Step 1: the cover encodings of x_1^n come from the previous iteration (or from Algorithm 1 at the start) and are split by the length of the string they decode to. Step 2: encodings longer than n bytes are regrouped by their $(n{+}1)$-th byte. Step 3: encodings of exactly n bytes are extended by next tokens predicted from $\mathrm{encode}(x_1^n)$ (one model call), and each candidate is accepted or rejected by the validity criterion. Step 4: both groups are combined into the cover encodings of each candidate string $x_1^n x_{n+1}$. Step 5: x_{n+1} is sampled from the resulting byte-level distribution. The probabilities $P(\vec{t})$ are omitted for clarity.

shared byte context, we form the ensemble by averaging,

$$P_{\text{ens}}(x_{n+1} \mid x_1^n) = \frac{1}{M} \sum_{m=1}^{M} P_m(x_{n+1} \mid x_1^n), \tag{6}$$

and sample or decode greedily from P_{ens}. Other combination rules over the common output space (e.g., weighted averaging or mixtures of experts) could replace Equation 6; we use the simple average throughout.

4 Experiments

4.1 Experimental setup

Models. For the synthetic study we use first- and third-order Markov chains over bytes, and a GPT-2 (Radford et al., 2019) architecture with 6 hidden layers trained on synthetic data from the third-order chain; since tokenization bias concerns tokenized models, the model operates on tokens and is queried with tokenized contexts. For code we use CodeLlama-7b (FIM experiments), CodeLlama-7b and CodeLlama-13b (listed as CodeLlama2-7b/13b in Table 2) (Rozière et al., 2023) and Yi-Coder-1.5B; for general tasks we use Llama2-7B (Touvron et al., 2023), Yi-1.5-6B (Young et al., 2024) and Mistral-7B-v0.3 (Jiang et al., 2023a). All are Transformer LLMs with their own tokenizers and are used without any weight update.

Tasks and metrics. FIM is evaluated on the HumanEval random-span infilling benchmark for Python (Bavarian et al., 2022; Chen et al., 2021) with the PSM and SPM prompt formats, reporting greedy pass rate and pass@10 (the maximum over a sweep of sampling temperatures). Code generation is evaluated with pass@1 on HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021); knowledge and reasoning with accuracy on MMLU (Hendrycks et al., 2021), PIQA (Bisk et al., 2020), Natural Questions (NQ) (Kwiatkowski et al., 2019), TriviaQA (Joshi et al., 2017) and GSM8K (Cobbe et al., 2021). Each token-versus-byte comparison keeps the model and prompts fixed and changes only whether predictions use the token-level distribution or the byte-level distribution of Section 3.5.

Baselines. We compare against standard token-level prediction, token healing (Dagan et al., 2024) and token alignment (Athiwaratkun et al., 2024), which are designed for prompts that end inside a token and are applied in SPM mode. For ensembles we compare against each member and, on MMLU and PIQA, where it was evaluated, against voting between the top-2 models. Ensembles combine the two best individual models on each benchmark ("top-2") by averaging their byte-level probabilities (Equation 6).

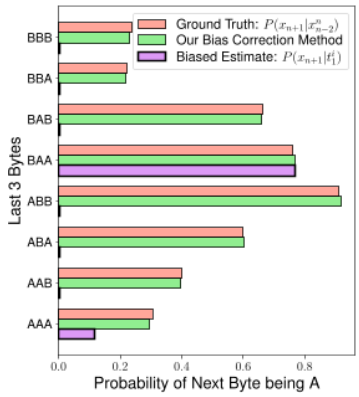


Figure 5: Tokenization bias and its correction on a third-order Markov chain. For each context of the last three bytes, bars give the probability that the next byte is `A`: ground truth $P(x_{n+1} \mid x_{n-2}^n)$ of the chain (red), the estimate obtained by conditioning a trained tokenized LM on its token history, $P(x_{n+1} \mid t_1^k)$ (purple), and the byte-level probability from our byte-token conversion applied to the same model (Section 3; green). The token-conditioned estimate is close to zero for six of the eight contexts, whereas the converted probabilities recover the ground truth.

4.2 Tokenization bias on Markov chains

On the first-order chain of Figure 2, conditioning on the token encoding of the context assigns probability 0.0 to the byte transition `A` $\rightarrow$ `A`, whose true probability is $1 - \alpha$. On the third-order chain, the tokenized GPT-2 model, when conditioned on a single encoding of the context, exhibits a severe bias in its predictive distribution relative to the ground-truth transition probabilities (Figure 5). Applying the byte-token conversion of Section 3.5 to the same trained model accurately recovers the original transition probabilities $P(x_{n+1} \mid x_{n-2}^n)$ for every three-byte context, without retraining.

4.3 Fill-in-the-middle code completion

Figure 6 reports pass rates of CodeLlama-7b on HumanEval random-span infilling. With standard token-level prediction, switching from the PSM to the SPM format lowers the greedy pass rate from 64.9% to 45.0% and pass@10 from 84.0% to 66.5%. In SPM the prefix is immediately followed by the generated middle, so a randomly placed span boundary inside a word leaves the model continuing from a partial token, an out-of-distribution tokenization. Byte-level prediction removes most of this degradation: its SPM greedy pass rate is 63.9% and its SPM pass@10 is 84.3%, improvements of 18.9 and 17.8 percentage points over token-level prediction that bring SPM within 1.8 points (greedy) of byte-level PSM. It also scores above both heuristic fixes in SPM mode, with 63.9% versus 62.7% (token healing) and 63.0% (token alignment) greedy, and 84.3% versus 83.8% and 82.9% at pass@10; these margins (0.5–1.4 points) are small, and no variance estimates are available to assess them. The temperature sweep in Figure 6 (right) shows that the token-level SPM curve stays well below PSM across temperatures, whereas the byte-level curves for PSM and SPM nearly coincide.

In the PSM format, where prompts are less affected by token boundaries, byte-level prediction is comparable to or slightly above token-level prediction: 65.7% versus 64.9% greedy and 84.1% versus 84.0% at pass@10. Inspecting individual failures of the token-level model, we find prompts that end in the middle of an identifier such as `delimeter`, `collatz` or `palindrome`: the token-level model fails to complete the word because of the token-boundary mismatch, whereas the byte-level method completes it correctly.

4.4 Byte-level prediction and ensembling across tokenizers

Byte-level versus token-level prediction. Tables 1 and 2 compare token-level and byte-level predictions of individual models. Over the 21 (model, benchmark) pairs, byte-level prediction is higher

Predictions	PSM Greedy	PSM @10	SPM Greedy	SPM @10
Token-level	64.9	84.0	45.0	66.5
+ healing	|	|	62.7	83.8
+ alignment	|	|	63.0	82.9
Byte-level	**65.7**	**84.1**	**63.9**	**84.3**

@10 here is the max over the temperature sweep on the right

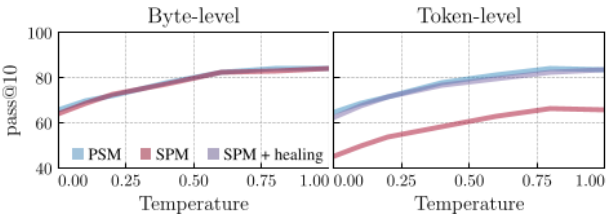


Figure 6: Pass rates (%) on the HumanEval Random Span infilling benchmark (Bavarian et al., 2022) with CodeLlama-7b, using the Prefix-Suffix-Middle (PSM) and Suffix-Prefix-Middle (SPM) prompt templates. Left: greedy pass rate and pass@10, where @10 is the maximum over the temperature sweep on the right. The baselines token healing (Dagan et al., 2024) and token alignment (Athiwaratkun et al., 2024) target prompts that end mid-token and are evaluated in SPM only (vertical bars). Right: pass@10 versus sampling temperature for byte-level and token-level predictions (the token-level panel also shows SPM with token healing). SPM forces predictions from the middle of a token: token-level greedy drops from 64.9 (PSM) to 45.0 (SPM), while byte-level predictions reach 63.9 in SPM, above both baselines, and are highest in every column.

Table 1: Accuracy (%) of general-purpose LMs with token-level and byte-level predictions on knowledge and reasoning benchmarks. Byte-level predictions are obtained from the same models without retraining. Voting and the byte-level ensemble combine the two best-performing models on each benchmark and are reported in the byte-level columns; token-level entries are not applicable (–). Voting was not reported for NQ, TriviaQA and GSM8K. Bold: best score per benchmark.

	MMLU		PIQA		NQ		TriviaQA		GSM8K	
Model	Token	Byte	Token	Byte	Token	Byte	Token	Byte	Token	Byte
Llama2-7B	45.4	45.7	78.1	78.2	25.0	23.4	58.4	58.1	15.1	12.5
Yi-1.5-6B	63.4	63.4	78.5	78.5	22.8	22.7	53.7	53.4	61.3	**61.5**
Mistral-7B-v0.3	62.1	62.1	80.1	80.3	28.5	28.8	63.6	63.5	39.0	39.0
Voting (top-2)	–	62.1	–	80.1	–	–	–	–	–	–
Top-2 Ensemble (ours)	–	**65.4**	–	**80.7**	–	**30.0**	–	**64.2**	–	55.8

in 7, equal in 4 and lower in 10, with differences between -3.6 and $+1.8$ points. On general benchmarks the two are close: the byte-level accuracy is within 0.3 points of the token-level accuracy on MMLU, PIQA and TriviaQA for all three models, and on NQ and GSM8K for Yi-1.5-6B and Mistral-7B-v0.3. The largest differences are decreases for Llama2-7B on NQ (-1.6) and GSM8K (-2.6). On code generation, byte-level prediction lowers HumanEval pass@1 for all three code models (-3.6, -2.3 and -1.9 points), while on MBPP it is higher for CodeLlama2-7b ($+1.8$) and Yi-Coder-1.5B ($+0.8$) and slightly lower for CodeLlama2-13b (-0.2). These benchmarks were not designed to contain mid-token prompts, so little gain is expected from removing tokenization bias. A possible explanation for the differences is that greedy decoding over bytes and over tokens selects different sequences, since the most probable byte at each step need not lie on the most probable token sequence, and that trained models satisfy Assumption 1 only approximately; we have not isolated these factors. Single-model byte-level prediction is therefore not a uniform improvement outside the mid-token setting; its value on these benchmarks lies in enabling ensembles.

Ensembling. Mapping models with different vocabularies into the byte space lets us average their predictions directly. The byte-level top-2 ensemble scores higher than the best individual model on MMLU (65.4 versus 63.4), NQ (30.0 versus 28.8) and, by smaller margins, PIQA (80.7 versus 80.3) and TriviaQA (64.2 versus 63.6); it also outperforms top-2 voting, which does not exceed the best member, on MMLU (65.4 versus 62.1) and PIQA (80.7 versus 80.1). For code, the ensemble of Yi-Coder-1.5B and CodeLlama2-13b reaches 42.1% HumanEval pass@1, 3.7 points above the best single model (Yi-Coder-1.5B, 38.4% token-level) and 5.6 points above its byte-level counterpart, although byte-level prediction lowers HumanEval pass@1 for each member individually. On MBPP the ensemble matches, but does not exceed, the best individual byte-level result (53.6%).

Table 2: Pass@1 (%) of code LMs with token-level and byte-level predictions. The byte-level ensemble combines the two best-performing models on each benchmark (–: not applicable). Bold: best score per benchmark.

	HumanEval		MBPP	
Model	Token	Byte	Token	Byte
CodeLlama2-7b	32.3	28.7	40.6	42.4
CodeLlama2-13b	35.8	33.5	47.6	47.4
Yi-Coder-1.5B	38.4	36.5	52.8	**53.6**
Top-2 Ensemble (ours)	–	**42.1**	–	**53.6**

Table 3: Effect of beam width on the byte-level FIM pass rate (%) on HumanEval. Width 1 matches the byte-level greedy results of Figure 6.

Beam width	PSM	SPM
1	65.7	63.9
2	66.8	65.9
4	**67.7**	**66.8**

Ensembling does not help when members differ widely in capability. On GSM8K, Yi-1.5-6B outperforms Mistral-7B-v0.3 by more than 20 points (61.5% versus 39.0% byte-level), and the ensemble (55.8%) falls below the stronger member. Averaging with equal weights lets the weaker model's errors override correct predictions of the stronger one; weighting members by validation performance would be a natural remedy that we have not evaluated.

4.5 Ablation: beam width

Table 3 reports the byte-level FIM pass rate on HumanEval random-span infilling as a function of beam width; width 1 coincides with the byte-level greedy results of Figure 6. Wider beams yield consistent improvements in both formats: from 65.7% to 67.7% (+2.0) in PSM and from 63.9% to 66.8% (+2.9) in SPM when going from width 1 to 4.

5 Conclusion

We identified tokenization bias, which can make a tokenized LM assign zero probability to correct continuations of prompts that end mid-token. The byte-token representation lemma expresses byte-prefix probabilities as a sum over cover encodings, and our backward search and recursive sampler compute the exact next-byte distribution of any tokenized LM at inference time, with one model call per byte after the prompt. The correction recovers the true transition probabilities of Markov chains, raises the SPM greedy pass rate of CodeLlama-7b on HumanEval random-span infilling from 45.0% to 63.9%, ahead of token healing and token alignment, and allows models with different tokenizers to be ensembled by averaging, which improves over the best member on most benchmarks we evaluated.

Limitations and future work. Our results also show the limits of the approach. Outside the mid-token setting, byte-level prediction from a single model is not uniformly better than token-level prediction and lowered HumanEval pass@1 for all three code models; equal-weight ensembles can hurt when members differ widely in ability, as on GSM8K. The method relies on the tokenized LM assigning negligible probability to invalid encodings, which holds approximately for trained models, and on access to the full next-token distribution. Generating byte by byte requires one model call per byte instead of per token, plus up to L calls to initialize the cover encodings of the prompt, which increases latency for long generations. Our FIM evaluation uses a single model and benchmark, and no seeds, repetitions or confidence intervals are available for any result, so differences below about one point (e.g., in PSM, against token healing and alignment, or the ensemble gains on PIQA and TriviaQA) should be read as comparable performance rather than improvements. The ensemble members are the two best models on each benchmark; if this choice uses test scores, the reported ensemble gains are optimistic. Future work includes reducing the per-byte cost by only invoking the model at candidate token boundaries, learned or performance-weighted ensemble combinations, and applying the byte-level view to other tokenizer mismatch problems such as speculative decoding with heterogeneous draft models.

REFERENCES

Mehdi Ali, Michael Fromm, Klaudia Thellmann, Richard Rutmann, Max Lübbering, Johannes Leveling, Katrin Klug, Jan Ebert, Niclas Doll, Jasper Schulze Buschhoff, et al. Tokenizer choice for LLM training: Negligible or crucial? *arXiv preprint arXiv:2310.08754*, 2023.

Ben Athiwaratkun, Shiqi Wang, Mingyue Shang, Yuchen Tian, Zijian Wang, Sujan Kumar Gonugondla, Sanjay Krishna Gouda, Rob Kwiatowski, Ramesh Nallapati, and Bing Xiang. Token alignment via character matching for subword completion. *arXiv preprint arXiv:2403.08688*, 2024.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

Mohammad Bavarian, Heewoo Jun, Nikolas Tezak, John Schulman, Christine McLeavey, Jerry Tworek, and Mark Chen. Efficient training of language models to fill in the middle. *arXiv preprint arXiv:2207.14255*, 2022.

Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 7432–7439, 2020.

Kaj Bostrom and Greg Durrett. Byte pair encoding is suboptimal for language model pretraining. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 4617–4624, 2020.

Kris Cao and Laura Rimell. You should evaluate your language model on marginal likelihood over tokenisations. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 2104–2114, 2021.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

Nadezhda Chirkova, Germán Kruszewski, Jos Rozen, and Marc Dymetman. Should you marginalize over possible tokenizations? *arXiv preprint arXiv:2306.17757*, 2023.

Jonathan H. Clark, Dan Garrette, Iulia Turc, and John Wieting. CANINE: Pre-training an efficient tokenization-free encoder for language representation. *Transactions of the Association for Computational Linguistics*, 10:73–91, 2022.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Gautier Dagan, Gabriel Synnaeve, and Baptiste Rozière. Getting the most out of your tokenizer for pre-training and domain adaptation. *arXiv preprint arXiv:2402.01035*, 2024.

Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Wen-tau Yih, Luke Zettlemoyer, and Mike Lewis. InCoder: A generative model for code infilling and synthesis. In *International Conference on Learning Representations*, 2023.

Juan Luis Gastaldi, John Terilla, Luca Malagutti, Brian DuSell, Tim Vieira, and Ryan Cotterell. The foundations of tokenization: Statistical and computational concerns. *arXiv preprint arXiv:2407.11606*, 2024.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021.

Yichong Huang, Xiaocheng Feng, Baohang Li, Yang Xiang, Hui Wang, Bing Qin, and Ting Liu. Ensemble learning for heterogeneous large language models with deep parallel collaboration. *arXiv preprint arXiv:2404.12715*, 2024.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7B. *arXiv preprint arXiv:2310.06825*, 2023a.

Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

Dongfu Jiang, Xiang Ren, and Bill Yuchen Lin. LLM-Blender: Ensembling large language models with pairwise ranking and generative fusion. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 14165–14178, 2023b.

Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1601–1611, 2017.

Taku Kudo. Subword regularization: Improving neural network translation models with multiple subword candidates. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 66–75, 2018.

Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 66–71, 2018.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.

Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. StarCoder: may the source be with you! *Transactions on Machine Learning Research*, 2023.

Benjamin Minixhofer, Edoardo Maria Ponti, and Ivan Vulić. Zero-shot tokenizer transfer. *arXiv preprint arXiv:2405.07883*, 2024.

Ivan Provilkov, Dmitrii Emelianenko, and Elena Voita. BPE-dropout: Simple and effective subword regularization. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 1882–1892, 2020.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Blog*, 1(8), 2019.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, et al. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

Phillip Rust, Jonas Pfeiffer, Ivan Vulić, Sebastian Ruder, and Iryna Gurevych. How good is your tokenizer? on the monolingual performance of multilingual language models. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 3118–3135, 2021.

Craig W. Schmidt, Varshini Reddy, Haoran Zhang, Alec Alameddine, Omri Uzan, Yuval Pinter, and Chris Tanner. Tokenization is more than compression. *arXiv preprint arXiv:2402.18376*, 2024.

Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1715–1725, 2016.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017.

Aaditya K. Singh and DJ Strouse. Tokenization counts: the impact of tokenization on arithmetic in frontier LLMs. *arXiv preprint arXiv:2402.14903*, 2024.

Yi Tay, Vinh Q. Tran, Sebastian Ruder, Jai Gupta, Hyung Won Chung, Dara Bahri, Zhen Qin, Simon Baumgartner, Cong Yu, and Donald Metzler. Charformer: Fast character transformers via gradient-based subword tokenization. In *International Conference on Learning Representations*, 2022.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Changhan Wang, Kyunghyun Cho, and Jiatao Gu. Neural machine translation with byte-level subwords. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 9154–9160, 2020.

Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning*, pp. 23965–23998, 2022.

Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306, 2022.

Alex Young, Bei Chen, Chao Li, Chengen Huang, Ge Zhang, Guanwei Zhang, Guoyin Wang, Heng Li, Jiangcheng Zhu, et al. Yi: Open foundation models by 01.AI. *arXiv preprint arXiv:2403.04652*, 2024.

Lili Yu, Dániel Simig, Colin Flaherty, Armen Aghajanyan, Luke Zettlemoyer, and Mike Lewis. MEGABYTE: Predicting million-byte sequences with multiscale transformers. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Yao-Ching Yu, Chun-Chih Kuo, Ziqi Ye, Yu-Cheng Chang, and Yueh-Se Li. Breaking the ceiling of the LLM community by treating token generation as a classification for ensembling. *arXiv preprint arXiv:2406.12585*, 2024.

Vilém Zouhar, Clara Meister, Juan Luis Gastaldi, Li Du, Mrinmaya Sachan, and Ryan Cotterell. Tokenization and the noiseless channel. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 5184–5207, 2023.

A Implementation of the byte-level sampler

This appendix describes how the procedure of Section 3 is carried out; it describes the method and contains no additional measurements.

Validity checks. A candidate $(\mathrm{encode}(x_1^j), t)$ is valid if and only if $\mathrm{encode}(x_1^j\, t) = (\mathrm{encode}(x_1^j), t)$ (Definition 1). The direct implementation re-encodes the string $x_1^j t$ and compares the result with the candidate token by token; because the history is already canonical, the candidate is valid exactly when the re-encoding reproduces the history unchanged and ends with the single token t. Each check costs one call of the tokenizer and no model call. In Step 3 of the sampler (Figure 4) the candidates are all tokens beginning with a given byte; their checks are independent and can be run in parallel.

Probabilities of the histories. Algorithm 1 needs $P(\mathrm{encode}(x_1^{n-i}))$ and $P(\cdot \mid \mathrm{encode}(x_1^{n-i}))$ for $i = 1, \ldots, \min(L, n)$. These histories are distinct token sequences, and $\mathrm{encode}(x_1^{n-i})$ need not be a prefix of $\mathrm{encode}(x_1^n)$ or of each other, so each requires its own forward pass; $P(\mathrm{encode}(x_1^{n-i}))$ is the product of the next-token probabilities along that pass. Whatever leading tokens the histories share are computed once: the forward passes reuse the key-value cache of the longest common prefix, and only the differing tail tokens are recomputed. During generation, the one model call per byte in Equation 5 is on $\mathrm{encode}(x_1^n)$. Its probability is already known, because $\mathrm{encode}(x_1^n)$ is the cover encoding of x_1^n whose last token ends exactly at position n, and, by Assumption 1, it consists of a canonical history $\mathrm{encode}(x_1^j)$, with $n - L \leq j < n$, followed by one token, where $\mathrm{encode}(x_1^j)$ was evaluated at an earlier step or by Algorithm 1; keeping the key-value caches of the last L histories therefore lets each call process a single new token.

Numerical stability. All probabilities are kept as log-probabilities. After each step we divide the probabilities of the retained cover encodings by $P(x_1^{n+1})$, i.e., we store $P(\vec{t})/P(x_1^{n+1})$, so that the quantities stay of order one and Equation 5 is a normalized sum.

Beam search and ensembles. For beam width B, each hypothesis is a byte string together with its cover encodings and cached states. At each step every hypothesis is extended by the bytes with positive probability under Equation 5, scored by its cumulative byte-level log-probability, and the B best extensions are kept; $B = 1$ is greedy decoding. For an ensemble, every member keeps its own cover encodings and caches for the shared byte string, computes its next-byte distribution independently, and the distributions are averaged (Equation 6) before choosing the next byte, which is then appended for all members.